

Exercícios de Fixação: Lógica e Estruturas Básicas

Preparação para Algoritmos de Ordenação

Estes exercícios focam na leitura de variáveis, manipulação de índices de listas e lógica condicional, servindo como base essencial antes de avançar para algoritmos como *Selection Sort* e *Quick Sort*.

Exercício 1: Leitura de Código (Laços e Condicionais)

Faça o teste de mesa do código abaixo e determine qual será o valor exato impresso na tela ao final da execução. Explique linha por linha como o valor da variável `x` é alterado durante o laço.

```
1 x = 0
2 for i in range(1, 6):
3     if i % 2 == 0:
4         x = x + i
5     else:
6         x = x - 1
7 print(x)
```

Exercício 2: Leitura de Código (Troca de Posições - *Swap*)

A troca de valores entre diferentes posições de uma lista é a mecânica principal da maioria dos algoritmos de ordenação. Analise o código abaixo e escreva qual será o estado exato da `lista` ao final da execução.

```
1 lista = [8, 3, 5]
2 aux = lista[0]
3 lista[0] = lista[1]
4 lista[1] = aux
```

Exercício 3: Desenvolvimento (Busca do Menor Elemento)

Escreva um algoritmo que receba uma lista de números inteiros e encontre o **menor** valor presente nela.

Restrição: Não utilize funções nativas de ordenação ou busca (como `min()` ou `sort()`). Construa a lógica manualmente utilizando um laço `for` e uma variável de referência.

Exercício 4: Desenvolvimento (Somatório Condicional em Lista)

Crie um programa que declare uma lista de números inteiros e calcule a soma **apenas** dos números que são pares. Ao final, imprima o resultado. Este exercício treina a capacidade de iterar sobre uma coleção e aplicar um filtro lógico simultaneamente.

Exercício 5: Desenvolvimento (Lógica de Particionamento Básico)

Escreva um programa que receba uma lista de números e defina uma variável `P` (pivô) com um valor qualquer. O programa deve percorrer a lista original e criar duas novas listas: uma contendo todos os números menores que `P`, e outra contendo todos os números maiores ou iguais a `P`. Ao final, imprima as duas listas.

Exercício 6: Leitura de Código (Laços Aninhados)

Em algoritmos de ordenação, é muito comum termos um laço de repetição dentro do outro. Analise o código abaixo e informe quantas vezes a palavra "Oi" será impressa na tela, e qual será o valor final da variável `contador`.

```
1 contador = 0
2 for i in range(3):
3     for j in range(2):
4         print("Oi")
5         contador = contador + 1
```

Exercício 7: Leitura de Código (Deslocamento de Elementos)

O código a seguir move elementos dentro de uma lista, uma operação comum quando precisamos abrir espaço para inserir um novo valor. Faça o teste de mesa e escreva como a `lista` ficará ao final da execução.

```
1 lista = [10, 20, 30, 40]
2 for i in range(3, 0, -1):
3     lista[i] = lista[i-1]
4 lista[0] = 99
```

Exercício 8: Desenvolvimento (Contagem de Ocorrências)

Escreva um algoritmo que receba uma lista de números e um valor alvo X . O programa deve percorrer a lista e contar quantas vezes o valor X aparece nela, imprimindo o total no final. Este exercício treina a busca linear combinada com uma variável contadora.

Exercício 9: Desenvolvimento (Verificação de Ordenação)

Crie um programa que receba uma lista de números inteiros e verifique se ela já está ordenada de forma crescente. O programa deve imprimir `True` se estiver ordenada e `False` caso contrário.

Dica: Percorra a lista comparando cada elemento com o seu sucessor (o elemento da próxima posição). Se algum elemento for maior que o seu sucessor, a lista não está ordenada.

Exercício 10: Desenvolvimento (Inversão de Lista com Trocas)

Escreva um algoritmo que inverta a ordem dos elementos de uma lista sem utilizar funções prontas (como `reverse()` ou fatiamento `[::-1]`). Você deve realizar a inversão utilizando a lógica de *swap* (troca), movendo os elementos das pontas em direção ao centro.